Freenet

A Peer-to-Peer Platform for Real-Time Decentralized Applications

Idempotent Commutative Monoids and Adaptive Small-World Routing

Ian Clarke

ian@freenet.org

https://freenet.org

October 3, 2026

Abstract

Freenet is a peer-to-peer platform for building real-time decentralized applications without global consensus. Its core abstraction is a *contract*: a WebAssembly module that defines both the public state of an application and the merge semantics used to reconcile replicas. The platform does not impose a single consistency model. Instead, each contract exposes an idempotent commutative merge operation (equivalently, a join-semilattice) and an application-defined summary/delta protocol for efficient synchronization.

Each contract has a location on a one-dimensional ring derived from the hash of its code and parameters, and is replicated on peers near that location. Requests are routed using a per-neighbor, distance-indexed performance model rather than topological distance alone. For private state, the platform provides *delegates*: local WebAssembly agents that hold secrets on the user's device, act on contracts on the user's behalf, and exchange messages with user interfaces and other delegates, each tagged with its sender's identity.

This architecture targets decentralized applications such as group chat, collaborative documents, social feeds, and reputation systems, where availability and application-defined convergence are preferable to global linearizability. The paper describes the architecture implemented in the freenet-core reference runtime, a method for testing contracts against the merge laws on states observed on the network, and a 24-hour snapshot of routing behavior on the deployed network (predating the current routing estimator). It also identifies which mechanisms are deployed, implemented but not yet broadly exercised, experimental, or not yet implemented.

1 The Problem

The modern web is built on infrastructure owned by a small number of companies. Group chat, social feeds, collaborative documents, identity, and reputation all run on it. So does the power to moderate, surveil, and remove the people who use it. Freenet is a peer-to-peer platform for providing the same kinds of services without those single points of control.

Two broad families of decentralized systems have been deployed at meaningful scale, and neither is a good substrate for interactive applications.

Content-addressed storage. The original Freenet (Clarke et al., 2001) and IPFS (Benet, 2014) provide name-by-hash access to immutable data. Applications must add their own mechanisms for mutable state. Chat rooms, document editors, and social feeds consequently implement consistency independently, often with different behavior across clients.

Blockchains. Ethereum (Wood, 2014) and successor smart-contract platforms provide programmable mutable state, but their consistency model is global linearizable consensus. Every state transition is ordered with every other transition, and replicated by every participating node. The latency, throughput, and replication cost that follow are reasonable for value-transfer and authoritative ledgers, but they are too expensive for the interactive applications that constitute the bulk of the web.

The gap between these is large. A chat room does not need a global linear order on every message; it needs replicas that converge to the same content. A reputation system does not need every reader to download every signed record; it needs efficient summary-driven sync. A collaborative document does not need consensus on each keystroke; it needs an algebra of merge that the application defines.

The platform is built around that observation. Applications carry their own merge semantics, expressed as sandboxed WebAssembly. The platform handles routing, replication, synchronization, transport, and sandboxing generically over whatever algebra the application supplies, alongside a separate primitive (the delegate) for private state and sensitive operations that should not be replicated. The result targets a class of decentralized applications that has historically been awkward to build on either content-addressed or blockchain substrates.

2 Design Thesis

The design separates state reconciliation from state propagation. Each application packages an idempotent, commutative merge operation on byte-string state as a WebAssembly module. The platform determines where that state lives, how peers locate it, how replicas exchange differences, how updates reach subscribers, and how secrets remain local. Its routing and synchronization mechanisms operate independently of the application-supplied state algebra.

This separation distinguishes the system from common approaches to decentralized state in three ways.

Application-defined rather than fixed CRDTs. Conventional Conflict-free Replicated Data Type (Shapiro et al., 2011) frameworks provide implementations of counters, sets, registers, sequences, and other predefined lattices. A Freenet contract supplies its own merge function, validity invariants, and tiebreak rules. This permits workload-specific state representations, but the runtime cannot statically verify their algebraic properties: a contract with a non-associative merge function will not converge. Because contracts are content-addressed, such a failure remains attributable to a specific module that users can reject.

Per-contract consistency instead of global consensus. Blockchain platforms globally order state transitions. Freenet provides eventual consistency independently for each contract: replicas converge without coordinating the order of their updates with the rest of the network. This supports interactive applications whose updates can be merged, but not applications that require a single global order.

Separate public and private state. A chat room's contents may be public while each user's signing key remains private. Contracts hold replicated public state; delegates hold device-local private state. A delegate acts on contracts through the local core and decides what to release according to the identity of each message's sender.

The remainder of the paper describes the primitives that follow from this thesis (Section 3), how updates move through the network (Section 4), how data is located (Section 5), how trust

is established and what it does not cover (Section 6), the current implementation status and open problems (Section 7), and the position in related work (Section 8).

3 Core Primitives

This section names the artifacts that appear in the rest of the paper and fixes notation.

3.1 Peers and the Ring

A *peer* is an instance of the Freenet Core running on some host. Each peer p has a *location* $\ell(p) \in [0, 1)$. Locations live on a one-dimensional ring with wrap-around distance

$$d(x, y) \;=\; \min\{|x - y|,\; 1 - |x - y|\}\,.$$

A peer's location is derived deterministically from its external network address (Section 6). Each peer maintains a set of *neighbor* connections, bounded between configurable minimum and maximum sizes. Neighbor relations are reorganized over time by the topology rule described in Section 5.

3.2 Contracts

A *contract* $\mathcal{C}$ is a pair $(\text{code}, \text{params})$, where code is a WebAssembly module conforming to the platform's contract interface and params is an opaque byte string fixed at instantiation. The contract's *key* is

$$k(\mathcal{C}) = H\big(H(\text{code}) \,\|\, \text{params}\big)$$

for a cryptographic hash H. The two-stage form lets a client that already holds $H(\text{code})$ derive the key for a new contract from new parameters without re-hashing the full WebAssembly module. The contract's *location* $\ell(\mathcal{C}) \in [0, 1)$ is derived from $k(\mathcal{C})$ in the same way as a peer location. Each contract has a *state* $\sigma \in \mathcal{S}_{\mathcal{C}}$, replicated across peers near $\ell(\mathcal{C})$ on the ring. The contract's code defines $\mathcal{S}_{\mathcal{C}}$, the validity predicate $\mathsf{valid}_{\mathcal{C}}$, the merge operation $\oplus_{\mathcal{C}}$, and the summary/delta synchronization functions (all detailed in Section 4). Contracts are public: their state is readable by any peer that can route to $\ell(\mathcal{C})$. Because contracts, delegates, and user interfaces are all addressed by hash, the platform needs no version negotiation or upgrade protocol: a new version is published under a new key, and the old key keeps working as long as anyone hosts it.

3.3 Delegates

A *delegate* is a WebAssembly module that runs on a single user's device, inside the local Freenet Core. It holds private *secret state* (sandboxed by the core) and responds to messages from user interfaces and other delegates. Each such message carries a sender identifier attached by the core. A delegate can also read, publish, subscribe to, and update contracts, and, with the user's consent, run on a schedule when no user interface is open. Delegates are local; they do not appear on the ring. Their trust boundary and the intended cross-device replication pattern are described in Section 6.

3.4 User Interfaces

A *user interface* is a web frontend (HTML, JavaScript, WebAssembly) delivered by a contract whose state encodes the application bundle. User interfaces run in the browser and communicate with the local core over a WebSocket API. They reach contracts and delegates only through

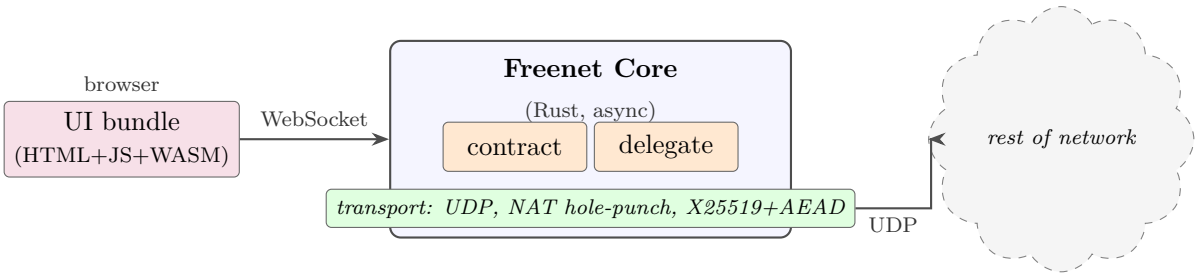


Figure 1: What runs in and around a single peer. The Freenet Core is a Rust process that hosts WebAssembly *contracts* (whose state is public and replicated across peers near the contract's ring location) and *delegates* (whose state is local to the device and never leaves it). A *user interface* is an HTML/JS/WASM bundle delivered as contract state and run in the browser; it talks to its local Core over a WebSocket. The Core's transport speaks UDP with NAT hole-punching to neighbors on the rest of the network.

that local core. They are not a distinct kind of network-visible artifact: from the platform's perspective, the bundle is just contract state with a particular MIME type.

3.5 Operations

A small set of operations connect these artifacts. We describe them informally here; their lifecycles appear in Sections 4 and 5.

PUT
Publish a contract: insert or update its state at the peers near $\ell(\mathcal{C})$.

GET
Retrieve a contract's current state by key.

UPDATE
Submit an update to a contract. The update is merged into existing replicas using the contract's merge function and propagated to the other peers hosting the contract.

SUBSCRIBE
Register interest in future state changes to a contract.

CONNECT
Establish a peering relationship with the network as a new node.

PUT, GET, UPDATE, and SUBSCRIBE all route over the ring; their destination is $\ell(\mathcal{C})$. CONNECT is the bootstrap operation by which a fresh peer joins the topology.

4 How Updates Move

This section describes the algebra each contract carries, the synchronization protocol that propagates state efficiently across replicas, how the platform tests contracts against both, the co-host mesh through which updates flow, and how subscriptions and eviction keep hosting aligned with demand.

4.1 Contract State as an Idempotent Commutative Monoid

Definition 1 (Contract state algebra)**.** *A contract $\mathcal{C}$ defines:*

1. *a* state space $\mathcal{S}_\mathcal{C}$ *(an abstract set; the wire encoding is a byte string and the contract code defines the encoding, which must be canonical, because the runtime compares states by their encoded bytes);*
2. *a* validity predicate $\mathsf{valid}_\mathcal{C} : \mathcal{S}_\mathcal{C} \to \{\bot, \top\}$*;*
3. *an* identity element $e_\mathcal{C} \in \mathcal{S}_\mathcal{C}$ *with* $\mathsf{valid}_\mathcal{C}(e_\mathcal{C}) = \top$*;*
4. *a binary* merge *operation* $\oplus_\mathcal{C} : \mathcal{S}_\mathcal{C} \times \mathcal{S}_\mathcal{C} \to \mathcal{S}_\mathcal{C}$*, which is associative, commutative, and idempotent, with* $e_\mathcal{C}$ *as identity.*

An idempotent commutative monoid is equivalently a join-semilattice; the partial order is $a \sqsubseteq b \iff a \oplus_\mathcal{C} b = b$. This is the algebraic structure of a state-based CRDT (Shapiro et al., 2011). The monoid presentation emphasizes the relevant operational property: peers can merge state at any time, in any order, with any multiplicity, and obtain the same result.

The validity predicate is checked at the boundary. A peer that receives a candidate state σ with $\mathsf{valid}_\mathcal{C}(\sigma) = \bot$ rejects it and never merges it into its local replica. Validity typically encodes integrity invariants beyond what the type system enforces, for example "every record in the log carries a valid signature by an authorized key."

An *update* is itself an element of $\mathcal{S}_\mathcal{C}$. When a peer holding state σ receives an update u, it computes $\sigma' = \sigma \oplus_\mathcal{C} u$, subject to validity. There is no separate notion of operation: the algebra of state is the algebra of change. All contract functions, including the synchronization functions introduced below, must be deterministic functions of their inputs.

Remark (Convergence)**.** *For a finite multiset of valid updates* $u_1, \ldots, u_n \in \mathcal{S}_\mathcal{C}$ *and a starting state* σ_0*, the merged result* $\sigma_0 \oplus_\mathcal{C} u_{\pi(1)} \oplus_\mathcal{C} \cdots \oplus_\mathcal{C} u_{\pi(n)}$ *is independent of the permutation* π *by associativity and commutativity, and idempotence makes duplicate delivery harmless. These are the sufficient conditions for the Strong Eventual Consistency theorem of Shapiro et al. (Shapiro et al., 2011): peers that have delivered the same set of valid updates converge regardless of order or repetition. Freenet applies this result to monoids supplied by individual contracts.*

4.2 Worked Examples

Monotone counter. $\mathcal{S} = \mathbb{N}$, $\oplus = \max$, $e = 0$. The partial order is the usual $\leq$.

Last-writer-wins register. $\mathcal{S} = V \times \mathbb{N} \times H$, where V is the value type, $\mathbb{N}$ a Lamport-style timestamp, and H a tiebreaker hash. Merge is lexicographic max on the (timestamp, hash) pair.

Signed log. $\mathcal{S}$ is a finite set of records, each record carrying a signature by an authorized key. $\oplus = \cup$, $e = \emptyset$. Validity rejects any state containing a record whose signature does not verify against a key listed in params.

Observed-remove set. $\mathcal{S} = (E, T)$ where E is a set of (elem, id) pairs and T a set of ids that have been tombstoned. $\oplus$ is pairwise union; the materialized set is $\{\text{elem} : (\text{elem}, \text{id}) \in E,\ \text{id} \notin T\}$.

Discarding state. A contract that discards part of a merged result must discard on a condition that further merges cannot reverse. A bounded signed log that keeps the most recent N records is sound if the records it keeps are determined by the merged set; if they depend on arrival order, the merge is no longer idempotent. More generally, validity may depend on the rest of the state, so a component invalid at σ can become valid at some $\sigma' \sqsupseteq \sigma$. Discarding it at σ

makes the result depend on the order in which the two arrived, and nothing revisits the decision later. The observed-remove set is sound because a tombstone is permanent, so materializing $E \setminus T$ discards on a condition that cannot change. A record dropped because its author has not yet been merged is unsound. The same rule applied to an author who has been tombstoned is sound, because that absence is final. Where the condition is transient, the contract must retain the component and derive its visibility from the current state, or reject the update in full so that its sender offers it again.

4.3 Summary/Delta Synchronization

Pairwise merge is sufficient for convergence but, on its own, would require peers to ship full state when reconciling. For non-trivial contracts this is prohibitive. The platform therefore extends the contract interface with three additional functions that allow peers to exchange only the difference between their states.

Definition 2 (Sync interface)**.** *In addition to the algebra of Definition 1, a contract provides*

$$\begin{aligned} \mathsf{summarize}_{\mathcal{C}} &: \mathcal{S}_{\mathcal{C}} \to \Sigma_{\mathcal{C}}, \\ \mathsf{getDelta}_{\mathcal{C}} &: \mathcal{S}_{\mathcal{C}} \times \Sigma_{\mathcal{C}} \to \Delta_{\mathcal{C}}, \\ \mathsf{applyDelta}_{\mathcal{C}} &: \mathcal{S}_{\mathcal{C}} \times \Delta_{\mathcal{C}} \to \mathcal{S}_{\mathcal{C}}, \end{aligned}$$

where $\Sigma_{\mathcal{C}}$ and $\Delta_{\mathcal{C}}$ are contract-defined byte-string types.

The summary type $\Sigma_{\mathcal{C}}$ is intended to be much smaller than full state. For a signed log it might be the set of record identifiers, or a Merkle root over them; for a counter it might be the counter's value; for a large observed-remove set it might be a Bloom filter, or a pair of Bloom filters over E and T.

For the two-step protocol below to be useful, the contract author must ensure that $\mathsf{getDelta}$ produces something that, when applied, dominates the other peer's state in the partial order. Because a delta may be applied after its recipient has merged further updates since publishing its summary, the requirement quantifies over every later state of the recipient:

Property 1 (Sync soundness)**.** *For all $\sigma_A, \sigma_B \in \mathcal{S}_{\mathcal{C}}$, let $s = \mathsf{summarize}_{\mathcal{C}}(\sigma_B)$ and $\delta = \mathsf{getDelta}_{\mathcal{C}}(\sigma_A, s)$. Then for all $\sigma'_B \sqsupseteq \sigma_B$,*

$$\mathsf{applyDelta}_{\mathcal{C}}(\sigma'_B, \delta) \oplus_{\mathcal{C}} \sigma_A = \mathsf{applyDelta}_{\mathcal{C}}(\sigma'_B, \delta).$$

That is, applying δ brings the recipient at least as far up the lattice as merging σ_A in full would have, even if the recipient has advanced since s was computed. A contract meets this by computing δ from what s shows the recipient to lack: the summary keeps δ small, and the lattice order keeps it applicable once the recipient has advanced. A delta whose effect depends on the recipient being exactly at σ_B is a patch rather than a delta in this sense, and out-of-order delivery will misapply it. The platform does not verify this property; the contract is responsible for it.

4.4 The Two-Step Sync Protocol

Two peers A and B reconcile their states for a contract $\mathcal{C}$ as follows:

1. A computes $s_A = \mathsf{summarize}_{\mathcal{C}}(\sigma_A)$ and sends s_A to B.
2. B computes $\delta_B = \mathsf{getDelta}_{\mathcal{C}}(\sigma_B, s_A)$ and sends δ_B to A. A applies the delta: $\sigma_A \leftarrow \mathsf{applyDelta}_{\mathcal{C}}(\sigma_A, \delta_B)$.

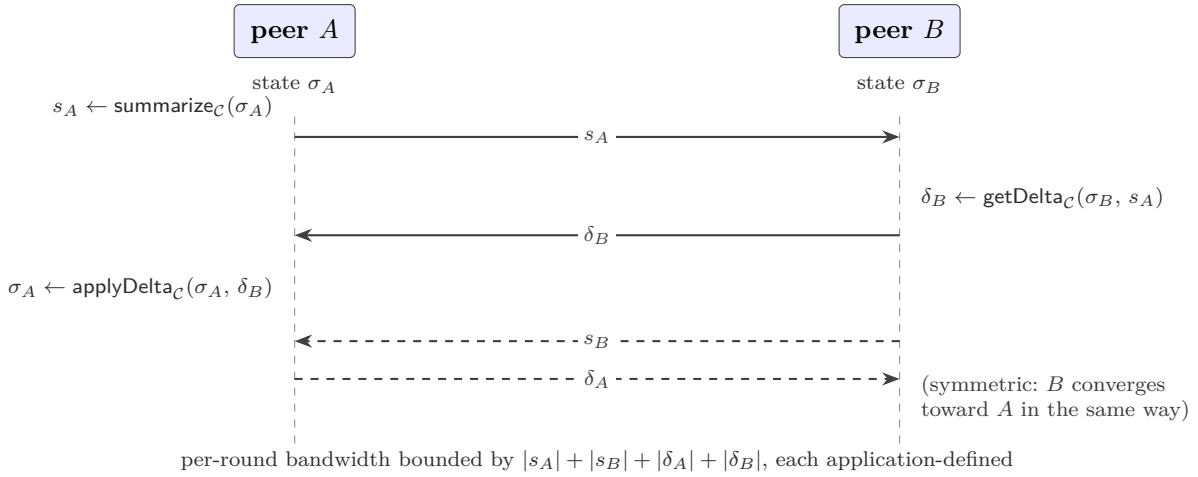


Figure 2: The two-step summary/delta sync protocol. Peer A sends a small summary s_A of its state. Peer B uses s_A to compute a *delta* relative to its own state and ships only the difference back, which A applies under the contract's merge. The exchange is symmetric (dashed); each direction runs over the same transport. The contract supplies $\mathsf{summarize}_{\mathcal{C}}$, $\mathsf{getDelta}_{\mathcal{C}}$, and $\mathsf{applyDelta}_{\mathcal{C}}$, so the protocol is generic over whatever state representation the contract uses.

The protocol is symmetric: B runs the same exchange in the other direction. In practice the two directions are interleaved over a single connection. Bandwidth per reconciliation round is bounded by $|s_A| + |s_B| + |\delta_A| + |\delta_B|$, each of which is application-defined. Contracts that use approximate summaries may require multiple rounds before replicas fully converge (see the next subsection), in which case the total bandwidth is the sum of these per-round bounds.

4.5 Malformed Summaries and Malicious Deltas

The basic protocol reconciles honest replicas with stale state. Approximate summaries and malicious peers require additional consideration.

Approximate summaries. A summary type such as a Bloom filter has false positives: B may believe A already has a record that in fact A does not. In the bare protocol, the record is then not included in the delta and the two replicas remain divergent. Contracts that use approximate summaries must therefore either fall back to a more precise second round when divergence is suspected, or include the false-positive rate in their convergence guarantees and rely on subsequent reconciliation rounds (driven by later updates, the periodic anti-entropy exchange described below, or explicit re-sync) to repair divergence. The platform offers no built-in fallback; the contract designs the round structure.

Malicious peers. A peer that supplies a deliberately malformed summary, or returns a delta that violates the validity predicate, is treated like any other source of invalid state. The receiver applies $\mathsf{valid}_{\mathcal{C}}$ to the merged result; if the result fails validity, the delta is rejected. A more subtle attack is for a peer to supply a delta that is valid but withholding (claiming to be up to date when it is not). The protocol cannot distinguish such a peer from one that has simply not yet received the missing updates; replication across several peers near $\ell(\mathcal{C})$ is the defense (Section 6).

4.6 Testing the Merge Laws

Fixed-lattice CRDT frameworks guarantee the algebraic laws by construction, because applications compose predefined types (Section 8). A Freenet contract defines its own lattice, so the runtime cannot verify the laws statically, and a contract that violates them fails to converge. Two mechanisms address this.

The first is containment. The runtime samples merges, reapplies identical inputs, and checks that the result reaches a fixpoint. When it detects non-idempotency it suppresses the contract's commits and outbound updates for an interval that lengthens on repeated detection. This bounds the damage of one class of violation; it establishes nothing about associativity, commutativity, or sync soundness.

The second is conformance testing, in the manner of property-based testing (Claessen and Hughes, 2000): a verifier executes the contract's own code on concrete states and checks that idempotence, commutativity, and associativity hold, and that every state the contract emits is valid. It also checks that the contract's functions are deterministic, that deltas applied in any order reach the same state, that the delta and full-state paths agree, and that simulated summary/delta rounds do not cycle, which approximates sync soundness (Property 1). Unlike conventional property-based testing, inputs are drawn from states observed on the network rather than generated at random. A case that the contract rejects, or cannot evaluate without other contracts' state, is reported as inconclusive and never as a violation. Contract authors run the verifier locally, and a node can run it against states sampled from the contracts it hosts, recording which contracts it would stop hosting without acting on the result. On one node's hosted set in September 2026 (55 contracts, 16,339 cases), it found 203 violations in seven contracts, including non-commutative merges in the River room contract that led to corrections in River's source.

Testing can find a violation but cannot prove its absence, and its findings are not yet enforced (Section 7).

4.7 The Co-Host Mesh, Subscriptions, and Hosting

Once a contract is published, each hosting peer advertises that fact to its connected neighbors, and these advertised links between *co-hosts* form the update mesh. An update arriving at any host is merged into local state and sent to its co-hosts: as a delta, where the sender has recorded a summary of the neighbor's state, and otherwise in full. Each update carries the list of peers it has already been sent to, and a co-host that forwards it skips them, which removes most of the duplicate copies that flooding an overlapping mesh would deliver. Because $\oplus_{\mathcal{C}}$ is commutative, simultaneous updates injected at different points of the mesh need no coordination; the order in which they reach a peer does not affect its final state.

Live propagation can miss a peer through a dropped message or a missed announcement. Neighbors that host contracts in common therefore also run a periodic anti-entropy exchange, which reconciles both their advertisements and their states. Each side sends a short digest of its summary for each shared contract, and the pair runs the summary/delta protocol only where the digests differ, so divergence that a contract's summary does not reflect goes undetected. Each round covers a bounded, rotating subset of the shared contracts, so its cost does not grow with the number of contracts two peers share.

Subscriptions express demand. A subscription is a soft-state path from an interested peer toward the contract's location; each peer on the path hosts and advertises the contract, so the path joins the same co-host mesh as every other replica. Subscriptions are leased, and a lease is renewed only while the contract is still *in use* at that peer, meaning it has a local client subscription, a downstream subscriber, or recent *genuine access* (a GET or PUT from a client, as

opposed to renewal traffic). A peer with no remaining interest stops renewing, its lease lapses, and the path shortens toward $\ell(\mathcal{C})$. Unused paths therefore expire within a small multiple of the lease period, and the set of live subscriptions tracks current demand.

Eviction bounds what each peer hosts. A peer sizes its hosting budget from its own resources, chiefly memory, because resident memory and update traffic limit the number of active contracts more than disk space does. When over budget, it evicts the contract with the least demand: fewest local client subscriptions first, then fewest downstream subscribers, then least recent genuine access. A subscription orders a contract later in this sequence but does not exempt it. A contract with no demand can also be evicted within budget if processing its updates consumes a large and sustained share of the peer's resources.

5 How Data Is Found

A request for contract $\mathcal{C}$ originates at some peer and must reach a peer that hosts $\mathcal{C}$'s state. Peer and contract locations and the ring metric are as defined in Section 3. This section describes the ring topology, the rule that constructs it and the bootstrap by which a new peer joins, how requests are forwarded and how the five operations use that forwarding, and the performance model that selects each hop. It closes with a 24-hour measurement of routing path lengths on the live network.

5.1 Why Small-World Routing

The routing strategy derives from Kleinberg's small-world result (Kleinberg, 2000). Consider N peers on a ring (or a lattice of any fixed dimension), each with a constant number of *short-range* links to immediate neighbors and a small number of *long-range* links sampled with probability proportional to $1/d$ in ring distance d. Kleinberg proved that a node with knowledge limited to its neighbors can greedily route to any target in $O(\log^2 N)$ hops. The result requires neither global topology knowledge nor flooding, provided the link distribution remains approximately $1/d$.

The $1/d$ distribution is what makes the bound work. With shorter long-range links, a hop does not reduce the remaining distance much; with longer ones, hops overshoot the target. With $1/d$, at every scale a peer has a constant probability of a link that halves the remaining distance, so each halving takes $O(\log N)$ expected steps and $O(\log N)$ halvings reach the target.

Kleinberg's analysis assumes the $1/d$ distribution. Symphony (Manku et al., 2003) sampled long-range links from a harmonic distribution at join time and retained them. Freenet instead adjusts each peer's neighborhood continuously. A peer directs new CONNECT requests toward under-represented regions of its current connection distribution (*gap-targeting*) and prefers incoming candidates that close large gaps (*Kleinberg-style scoring*). These mechanisms maintain the distribution under churn.

5.2 Topology Construction

The CONNECT operation constructs the small-world topology. A joiner chooses a *desired location*, routes a CONNECT message toward it, and receives acceptance decisions along the route and at its terminus. Target selection and acceptance both shape the topology toward Kleinberg's $1/d$ distribution (Kleinberg, 2000).

Desired location. A peer with only a few connections targets its own location, with random jitter after repeated acceptance failures so that later attempts probe different parts of the ring.

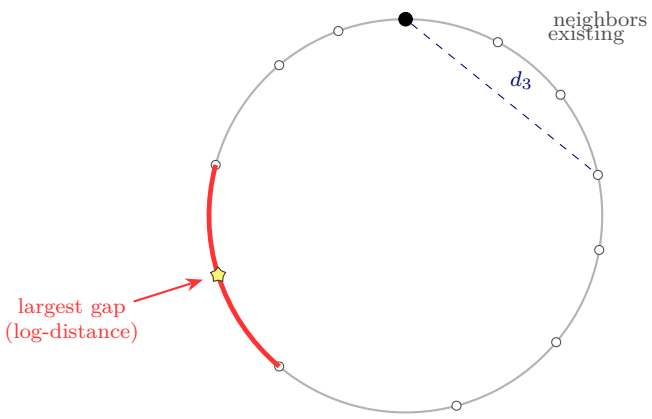


Figure 3: Gap-targeting from a single peer's view. The peer measures ring distance d_i to each existing neighbor (one such d_i shown dashed), identifies the largest gap in log-distance space (red arc), and chooses its midpoint as the next CONNECT destination (star). The peer thus pulls its own connection distribution toward $1/d$ as it accumulates neighbors.

Because joiners' locations differ, their early targets differ too, and bootstrap traffic spreads across the ring. A peer with more connections instead computes a *gap target*: it measures the ring distance from itself to each existing neighbor, finds the largest gap in that distribution in log-distance space, and targets the gap's midpoint. The locations under-represented by its current connections are thus exactly the ones its next CONNECT targets, and a peer with mostly local neighbors, whose largest gap is at long range, next connects to a distant peer. This is how each peer pulls its own connection distribution toward $1/d$.

Acceptance. A peer that receives a CONNECT forwards it to a neighbor no farther from the target than itself, if it has one. A peer within a small band around the target may also accept the joiner, with a probability that grows with proximity, while still forwarding; this gives the joiner more than one nearby connection per request.

At the terminus, the decision depends on the receiver's connection count. A peer with very few connections accepts unconditionally, so that bootstrap can proceed. Below its minimum it scores the candidate by how well the candidate's location fills gaps among its existing neighbors, accepting more selectively as it fills. At or above the minimum it admits only the best-scoring candidate among recent arrivals, and above the maximum it rejects. If the terminus rejects, the CONNECT may be forwarded a few hops away from the target, so that a joiner whose target lies behind a saturated peer can still be accepted by a nearby peer with capacity.

Nearest-neighbor lattice. Kleinberg's bound also assumes a short-range base lattice. Each peer therefore seeks and keeps its nearest successor and predecessor on the ring. Maintenance CONNECTs routed toward the peer's own location discover closer candidates on either side; a closer candidate is admitted even when the peer is at capacity, and pruning never discards the closest edge on either side. The remaining connections form the long-range distribution described above.

Joining and NAT traversal. Most peers are behind consumer NAT and have no stable inbound address. Two peers connect by hole-punching: both send hello packets toward each other's externally observed addresses at the same time (Ford et al., 2005). The *introducer*, which reports each side's observed address to the other, is whichever peer routed the CONNECT between them. For a new peer this is a gateway, a peer with a public address; for a peer that

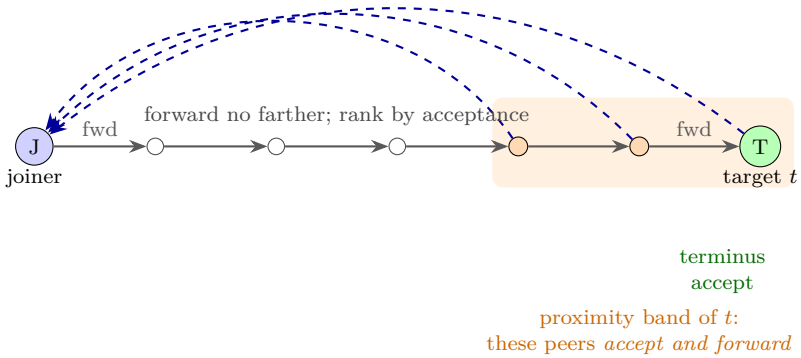


Figure 4: A single CONNECT from joiner J to its desired location t. Each hop forwards only to neighbors no farther from t than itself, choosing among them with an estimator trained on past acceptance outcomes (solid arrows). Once the route enters a small band around t (shaded), each receiver may also accept the joiner, with a probability that grows with proximity, while still forwarding the request (dashed return edges). One CONNECT can therefore establish several connections close to t.

already has a connection, it is one of its neighbors. Gateways are therefore needed only for the first connection, and the network continues to form connections while every gateway is offline.

5.3 Forwarding

A request from peer p to target location t proceeds as follows. Let $\Gamma(p)$ be p's current set of neighbors, excluding peers the request has already visited. A peer without enough routing history orders $\Gamma(p)$ by distance to t, placing first the neighbors it has not yet routed through so that it gathers evidence about each of them. Once it has that history, it takes a fixed number of the neighbors closest to t as candidates and selects among them by the predicted cost of Section 5.5. That number is close to a peer's minimum connection count, so distance acts mainly through the cost model rather than as a filter. Kleinberg's $O(\log^2 N)$ bound (Section 5.1) is proved for strict greedy routing, in which every hop moves closer to the target; we do not claim it for this cost-ranked form.

Each request carries a *hops-to-live* (HTL) counter, deployed maximum 10, that decrements on each forward: when it reaches zero, the current peer stops forwarding and answers from local state or reports a miss. A peer that is near t but does not host the contract forwards to its next-best candidate, and visited peers are never revisited.

5.4 Lifecycles

The five operations of Section 3 are realized as transactions, each driven to completion by its own task in the local core, with timeouts, retries, and parent/child composition.

CONNECT. As above. Each successful CONNECT establishes one neighbor relationship. CONNECTs are issued by a maintenance loop, not as a side effect of ordinary request traffic.

PUT. A PUT carries a contract $\mathcal{C}$ and an initial state σ_0. The originator validates and stores the state, computes its summary, and routes a probe toward $\ell(\mathcal{C})$; intermediate peers that do not host the contract forward the probe without storing anything. If the probe reaches a peer that already hosts the contract, that peer returns its summary, and the originator sends it the

corresponding delta, which then propagates through the co-host mesh like any update. Otherwise the operation falls back to sending the full state, which the peers along the route validate and host. A PUT ends when its route has descended toward $\ell(\mathcal{C})$ and no remaining candidate is closer.

GET. A GET carries a key k and is routed toward the corresponding location. Any peer on the path that hosts the contract may answer with its current state and parameters, and the response returns along the request path. A route that ends without finding a host does not immediately report a miss: the last peer consults its neighbors' host advertisements (Section 4) and forwards to a neighbor that hosts the contract, if one exists. Forwarding reaches the right part of the ring, and the advertisements supply the final step. A retried GET avoids first hops that already reported the contract missing.

The requester can verify that the served code and parameters hash to k, but not the state: the key is derived from code and parameters, and state is mutable. This differs from immutable content-addressed stores such as IPFS, where the value returned for k itself hashes to k. State is trusted only to the extent that the contract's validity predicate accepts it (Section 6).

UPDATE. An UPDATE carries a key k and a candidate state $u \in \mathcal{S}_{\mathcal{C}}$ and is routed toward $\ell(\mathcal{C})$. Each peer on the path that hosts the contract merges u into its copy, subject to the validity check, and sends the change to its co-hosts (Section 4).

SUBSCRIBE. A SUBSCRIBE is routed toward $\ell(\mathcal{C})$ and ends like a GET; the peers along the path record the subscription and host the contract, forming the leased demand path of Section 4.

5.5 Adaptive Routing

Ring distance alone does not identify the fastest path because neighbors may differ by an order of magnitude or more in bandwidth, RTT, and uptime. The router therefore selects the next hop by predicted total cost.

The local peer maintains a history of routing events of the form (q, d, c, τ, b, f), where q is the selected neighbor, d is its ring distance to the request's target, c is the contract's location, τ is the observed time-to-first-byte, b is the observed transfer rate, and f is a failure indicator. A request that ends in a miss or a timeout counts as a failure of the route that served it, so the failure signal covers the neighbor's onward path as well as the neighbor itself. From this history the router predicts three outcomes for each candidate:

$$\widehat{\tau}_q(d), \quad \widehat{p}_q(d), \quad \widehat{b}_q(d),$$

the expected latency, failure probability, and throughput. Failure is modeled as a probability, and latency and throughput on a logarithmic scale. Each prediction has two parts: a distance curve $g(d)$ shared by all neighbors, and a hierarchy of corrections specific to the neighbor and the part of the ring the contract lies in.

Distance prior. The shared curve g is fit by *isotonic regression* (Barlow et al., 1972; de Leeuw et al., 2009), the standard non-parametric estimator for a monotone function. The choice of isotonic regression encodes a structural prior we are willing to commit to (latency and failure are non-decreasing in distance-to-target, on average) without committing to a parametric family for the shape of the curve. The pool-adjacent-violators fit is shrunk toward the pooled mean, so that a block supported by few events cannot dominate it, and then re-fit to restore monotonicity.

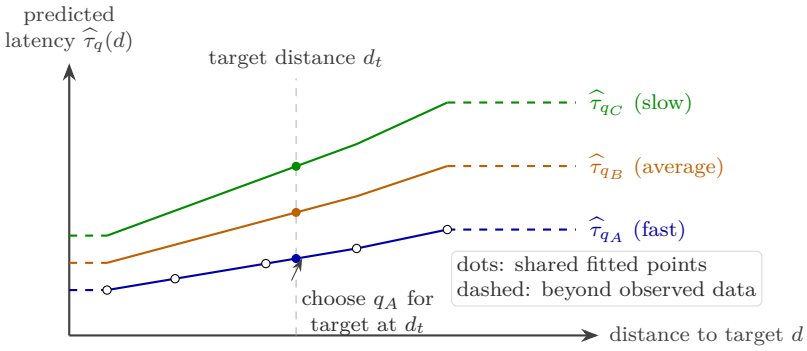


Figure 5: Per-neighbor latency predictions used by adaptive routing. Each neighbor q's prediction $\widehat{\tau}_q(d)$ is a non-decreasing function of ring distance to the target: one distance curve, fit by isotonic regression over all neighbors' routing events, scaled for each neighbor by a factor that is shrunk toward the network-wide value in proportion to how little evidence supports it. Dots mark the shared fitted points; beyond the observed distances each curve holds its end value. Two further predictions track failure probability and throughput; one outcome is shown for clarity. For a request to a target at distance d_t, the router reads each neighbor's predictions at d_t and picks the neighbor that minimizes the combined expected cost. The chosen neighbor need not be the one topologically closest to the target.

Hierarchical corrections. Neighbors differ from the shared curve in ways that persist: one has a slow uplink, another selectively drops requests for one part of the key space. The router models the residual $r = y - g(d)$ of each observed outcome y as a sum of effects at three levels: a network-wide offset, a per-neighbor offset, and a per-neighbor offset for each of several fixed arcs of the ring, assigned by contract location. A prediction descends this hierarchy: at each level the estimate moves from its parent's value toward the level's own mean by the fraction $P/(P+V)$, where P is the variance of effects at that level and V the sampling variance of the level's own mean. Both variances are estimated from the data, in the manner of parametric empirical Bayes (Morris, 1983). A neighbor observed twice therefore moves its prediction little, while one observed a thousand times is predicted mostly from its own record.

Forgetting. Each level discounts older events exponentially. The rate of discounting is selected online: the router keeps one hierarchy for each of several rates and predicts with whichever has the lowest recent loss on outcomes it forecast before observing them. Onward-path quality changes rapidly under churn because it depends on peers beyond the immediate link, and this lets the model track it without fixing in advance how quickly it changes.

Contract effects. A failure on a contract that no peer can serve is not evidence about the neighbor that was asked. If learned as such, it would raise that neighbor's predicted failure rate for every other contract. The failure model therefore adds a per-contract effect, estimated from the outcomes of *other* neighbors on the same contract, and subtracts it from what the neighbor-level terms learn. A neighbor that fails on every contract is still charged with its own failures, because a neighbor's own outcomes never contribute to the contract effect subtracted from its record.

Selection. Given a target at distance d_t from candidate q, the router selects the candidate that minimizes the expected cost

$$\widehat{\tau}_q(d_t) + \bar{s}/\widehat{b}_q(d_t) + 3\,\widehat{\tau}_q(d_t)\,\widehat{p}_q(d_t),$$

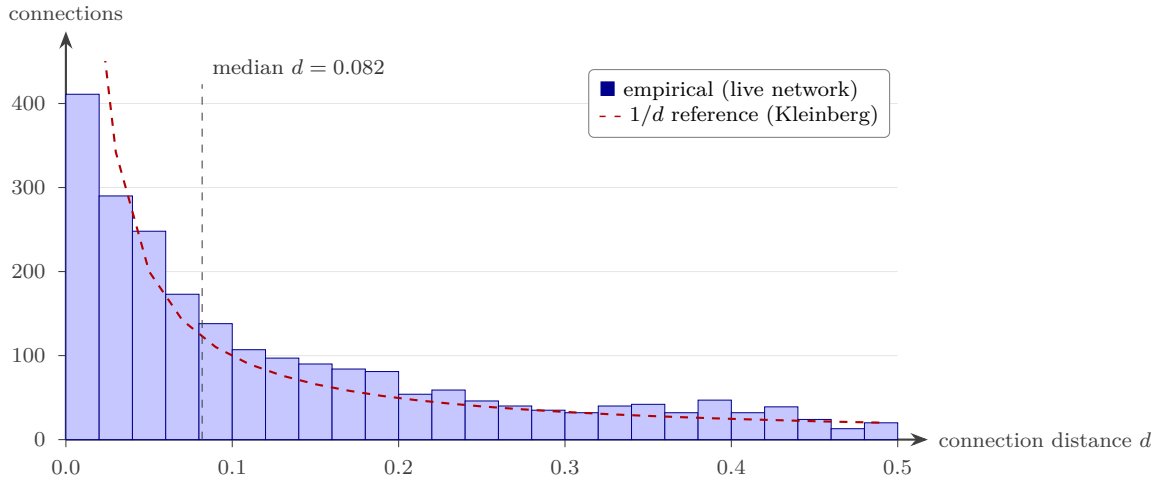


Figure 6: Distribution of per-connection ring distances on the deployed network: $n = 2274$ live connections across $N = 396$ active peers in a snapshot taken on 2026-05-25, in linear bins of width 0.02. The dashed curve is a Kleinberg $1/d$ reference normalized to the same total mass. The distribution decreases monotonically in d, concentrated at short range with a long-range tail. The median connection distance is 0.082.

where $\bar{s}$ is the mean transfer size, so a failure is charged as roughly three further response times. Timing is observed only on successful transfers, and a candidate without a timing estimate is ranked by $3\,\widehat{p}_q(d_t)$ alone. The timing predictions account for the uncertainty in each neighbor's own estimate, so a neighbor with little history is predicted to be slower than an established one with the same typical performance. While an outcome's hierarchical estimate lacks data, the router uses a simpler estimator for it.

Because the model consumes end-to-end routing outcomes, its predictions combine the neighbor's link quality (RTT, bandwidth, and reliability) with the quality of onward paths through that neighbor.

CONNECT does not use this model. It forwards only to neighbors no farther from the target than the current peer, ranked by a separate estimator of acceptance, so that topology-construction traffic keeps moving toward its target.

5.6 A 24-Hour Snapshot of Routing on the Deployed Network

This subsection records a single observation of routing on the deployed network, as a consistency check rather than a scaling study. It predates the estimator described above: requests were then ranked by an earlier model that blended the isotonic distance curve with a k-nearest-neighbor predictor (Clarke, 2026). All peers ran with telemetry enabled, so every hop was observed. The 24-hour window ends 2026-05-24; 443 peers were active in its last 10 minutes and 604 distinct peers were seen over the whole window, the difference being churn. A snapshot of connection distances taken the following day (Figure 6) shows the qualitative small-world shape the topology rule is designed to produce. It is more heavy-tailed than a pure $1/d$ law, which is not in itself better: Kleinberg's bound depends on the tail exponent, and relating the tail exponent to hop count requires the measurements across network sizes discussed below.

For each GET and PUT transaction we compute a *path length*: the number of distinct peers seen emitting the corresponding *_request event for that transaction id, which is the number of peers visited (hops +1). Statistics over operations whose telemetry reached completion in the

window:[1]

Operation	n	mean	median	p95	p99
GET	1,302	6.84	7	11*	11*
PUT	158	6.01	7	10*	10*

**The hops-to-live cap of 10 drops any route that would visit 12 or more peers before it can appear in the table, so these percentiles mark where the tail is truncated; the upper tail itself is not observed.*

The measurement has two further limitations. First, it covers one observation at one network size. At $N \approx 443$, $\log_2 N \approx 8.8$; the observed median path length of 7 peers (6 hops) for GET is consistent with logarithmic-diameter routing, but a single data point at a single N cannot distinguish $O(\log N)$ from $O(\sqrt{N})$, $O(N^{0.4})$, or a constant. Second, the table does not compare against any baseline, so it does not isolate the contribution of adaptive routing from distance-ordered forwarding. Both require measurements across orders of magnitude in N, which remain open (Section 7).

6 Security and Trust Boundaries

The platform has several layers, each with distinct authentication and trust assumptions. This section defines their guarantees, boundaries, and excluded threats.

6.1 Transport: Hop-by-Hop Encryption and Authentication

A connection between two neighboring peers is established by an authenticated ephemeral Diffie-Hellman key exchange, after which traffic is protected by authenticated symmetric encryption. The initial hello is encrypted to the recipient's long-term identity key, so a peer does not respond to a hello it cannot decrypt, and its port is invisible to a passive scanner.

This authentication is *hop-by-hop*. A request that traverses peers $A \to B \to C \to D$ is authenticated by A to B, by B to C, and by C to D. There is no transport-level guarantee that the message body at D is the body A originally sent: an intermediate peer can drop, reorder, or modify the payload before forwarding.

6.2 Application Layer: End-to-End Authentication via Validity Predicates

End-to-end integrity is the contract's responsibility. A contract's validity predicate valid_C runs at every peer that touches the state. Update messages typically include a signature by an application-authorized key over the update content; the validity predicate verifies the signature and rejects the update otherwise. Because valid_C is itself part of the contract's content-addressed code, a peer cannot serve a state that fails validity without being caught by every receiving peer that runs the same code.

The combination is the platform's substitute for a single end-to-end transport channel. Transport gives hop-by-hop confidentiality and authentication of neighbors. The contract gives end-to-end semantic integrity of state.

[1] In this May 2026 snapshot, UPDATE (median path length 2, $n = 386$) and SUBSCRIBE (median 1, $n = 5{,}359$) propagated primarily through pre-existing subscription paths rather than fresh routes, so their path lengths reflected the distance from the subscriber to the nearest hosting peer rather than routing-layer hop counts. They are excluded from the table here.

6.3 Location Assignment: A Known Sybil and Grinding Surface

A peer's location is a hash of its externally observed network address, taken over the address *prefix*: the /24 of an IPv4 address, so all hosts in the same /24 share a location, and a site-level prefix of an IPv6 address. This raises the cost of location grinding, since an adversary who wants peers at chosen ring locations needs many distinct prefixes rather than many hosts in one subnet.

Prefix hashing only raises that cost. An adversary who can rent address space across many /24s, obtain an IPv6 range, or provision machines across distinct cloud prefixes can still concentrate peers in chosen ring regions, then selectively answer requests for contracts there or crowd honest peers out of the topology.

Prefix hashing also places every honest peer sharing a prefix at the *same* location: independently operated peers behind a university or corporate NAT, a carrier-grade NAT pool, or a popular cloud region have ring distance zero from one another. This affects three things. *Topology construction:* gap-targeting (Section 5) assumes neighbor locations spread around the ring, and a cluster of co-located peers is a zero-width spike to it; and an adversary able to grind locations on both sides of a target can occupy its predictable nearest-neighbor slots and attempt a local eclipse, although failed links are still removed. *Replication:* peers near a contract's location are meant to protect against a single peer serving stale or doctored state, but co-located peers may share an operator and failure domain. *Sybil defenses:* the application-layer defenses below depend on honest peers being diverse near critical locations, which shared prefixes make less likely. Adding a per-peer nonce would spread co-located peers but would restore the grinding freedom that prefix hashing removes.

The platform does not solve this at the location layer; two mitigations operate above it. First, validity predicates make forged state detectable: a state with an invalid signature is rejected by every honest receiver, so the remaining attack is to serve stale but valid state. Second, identity, reputation, and admission control are expected to be built as application-layer contracts (Ghost Keys (The Freenet Project, 2026a) is an example of a contract-mediated anonymous identity scheme). A contract that publishes authoritative information can require its writers to hold reputation tokens issued by another contract; an inbox contract can demand a small proof of work; a forum can require introductions from existing members. A platform-level scheme would either impose a particular economic or identity model or provide a generic obstacle that a well-resourced adversary could overcome.

These application-layer contracts are, however, routed and replicated through the same location layer. An attacker with enough prefixes to crowd a contract's location can serve stale reads of it and keep honest writes from reaching its replicas. Bootstrapping these defenses therefore needs a property the platform does not yet provide: diversity among the peers near safety-critical contracts, separation of their replicas on the ring, or an out-of-band trust anchor such as a bundled root key.

6.4 Delegates: A Trust Boundary Around Secrets

Applications also have private state (a user's signing keys, a private contact list, the decryption material for an encrypted inbox) that should never be replicated and should be accessible only through controlled interfaces. A delegate (Section 3) holds such state inside the local Freenet Core, and user interfaces and other delegates reach it only by sending messages. For a message from a user interface, the sender identifier is the contract that delivered that interface; for a message from another delegate, it is that delegate's key, a hash of its code and parameters that different code cannot claim. A delegate's policy can therefore be "respond only to the user interface delivered by contract X" or "respond only to delegate Y."

Traditional in-process encapsulation (a "private" field on an object) is honored only by code that chooses to honor it, since the same address space gives access to memory, the file system, and undocumented interfaces. A delegate's secret state is not addressable from any other code on the device; the only way to obtain a secret is to convince the delegate, on the basis of sender identity and message contents, that releasing it is allowed. The boundary is enforced by the local Core, which is itself software on the same host, so a compromised Core can violate it. Defending against that would require hardware-attested execution, which the platform does not provide; the trust model assumes an uncompromised local Core.

Worked example: signing without exporting a key. A chat application's UI needs to send signed messages. Without delegates, it must hold the private key. With delegates, the UI sends a (sign, payload) message to a key-manager delegate; the delegate verifies the sender, signs the payload, and returns the signature. The private key never crosses the boundary. A bug or compromise in the UI can produce unwanted signed payloads but cannot exfiltrate the key itself; the delegate is also free to apply rate limits or user confirmation prompts before signing.

Acting on contracts. A delegate can issue the same GET, PUT, and SUBSCRIBE requests as a user interface, and they are routed through the network in the same way; it can also submit UPDATEs to contracts its node already holds. When a contract it subscribes to changes, the local core notifies the delegate, and these subscriptions persist across restarts of the node. A delegate can therefore maintain an application's public state on the user's behalf, for example by publishing signed updates to a contract it subscribes to, without exposing the signing key to any user interface.

Background execution. With the user's consent, a delegate may also run when no user interface is open: when the node starts and at periodic wake-ups, under a per-delegate budget of execution time and contract operations. An invocation that no user interface initiated cannot message other delegates.

Cross-device replication. A user typically runs the "same" delegate on several devices. The intended pattern for keeping those instances in sync is a shared-secret contract: a contract holding an encrypted, replicated copy of the shared private state, decryptable only by delegates that hold the key. The contract's merge function provides convergence and the delegate provides confidentiality. This pattern is not yet implemented (Section 7); today a user moves a delegate's secrets between devices by exporting and importing an encrypted bundle. The capabilities above nevertheless supply what such a delegate needs: it can subscribe to the shared contract, be notified of changes made on another device, and merge them without a user interface being open.

6.5 Resource Accounting and Its Limits

Each peer rate-limits its transport bandwidth and the updates it accepts from each neighbor for each contract, and disconnects a neighbor that consumes excessive resources. Each WebAssembly invocation runs under a time limit and a memory limit enforced by Wasmtime (Bytecode Alliance, 2026), and the set of hosted contracts is bounded as described in Section 4.

These limits protect a peer against a single misbehaving neighbor, not against coordinated traffic from many neighbors or denial of service through many cheap identities. Addressing those attacks probably requires accounting that persists across identities (Section 7).

7 Implementation Status and Open Problems

This section describes the reference implementation, reports the status of each major mechanism, identifies the applications the platform suits and those it does not, and lists unresolved questions.

7.1 Implementation

The Freenet Core is an asynchronous runtime written in Rust. Contracts and delegates execute in Wasmtime (Bytecode Alliance, 2026) under time and memory limits, so that a runaway or malicious module cannot exhaust the host, and contract state is persisted locally. Each operation runs as its own asynchronous task, so a peer's throughput is not limited to what a single event loop can sustain.

Deterministic simulation testing. A peer-to-peer system is hard to test by ordinary means: many bugs in routing, congestion, or convergence appear only in specific multi-peer interleavings that are difficult to reproduce. The Core ships with a deterministic simulation harness built on Turmoil (Tokio Project, 2026): virtual time, a seeded global RNG, an in-memory socket implementation, and fault injection. Tests run the entire network in a single deterministic process and replay bit-for-bit from a seed. Most non-trivial routing, subscription, and contract-execution bugs we have found were first reproduced in simulation.

7.2 Implementation Status of Each Mechanism

The following table reports the current status of the mechanisms described in this paper. "Deployed" means present in the production runtime and exercised on the live network; "Implemented" means present in the runtime but not yet broadly exercised; "Experimental" means available as an option but not used in production; "Open" means designed or partially designed but not implemented.

Mechanism	Status	Notes
Small-world ring topology	Deployed	Nearest-neighbor lattice plus gap-targeted long links
CONNECT targeting and acceptance	Deployed	Gap-targeted; score-based acceptance at terminus
Request forwarding	Deployed	HTL-bounded; cost-ranked once history exists
Adaptive routing	Deployed	Isotonic distance prior with hierarchical empirical-Bayes corrections
Summary/delta synchronization	Deployed	Per-contract, application-defined
Summary-first PUT	Deployed	Delta to an existing host; full-state fallback
Leased subscriptions	Deployed	Renewed only while the contract is in use
Co-host mesh and anti-entropy	Deployed	Live fan-out plus periodic digest comparison
Demand-ordered eviction	Deployed	Memory-derived budget
Non-idempotency containment	Deployed	Temporary suppression after detection
Merge-law conformance testing	Implemented	Local and opt-in on nodes; findings not enforced
Contract WebAssembly execution	Deployed	Wasmtime, time and memory limits
Delegates (local, secret state)	Deployed	Used by River chat (The Freenet Project, 2026b)
Delegate contract operations	Implemented	Requests, change notifications
Delegate background execution	Implemented	User-consented, periodic
Delegate cross-device sync	Open	Manual encrypted export and import today
NAT hole-punching transport	Deployed	UDP, authenticated encryption
Congestion control	Deployed	Fixed-rate default; LEDBAT++ and BBR experimental
Deterministic simulation testing	Deployed	Turmoil-based (Tokio Project, 2026)
Per-peer resource limits	Deployed	Bandwidth and per-neighbor update rate
Sybil-resistant peer identity / location	Open	Defenses are application-layer (Section 6)
Network-level incentive / abuse layer	Open	Application-layer reputation contracts intended

Congestion control is the least mature component. The deployed default is a fixed-rate token bucket; LEDBAT++ and BBR variants exist as alternatives, but neither has yet been adapted satisfactorily to peer-to-peer links of widely varying capacity.

7.3 Application Fit

Contract-defined merge, summary/delta synchronization, small-world routing, and delegates suit applications whose conflicts can be resolved by application-defined merge, such as group chat, collaborative documents, social feeds, reputation systems, asynchronous inboxes, and identity directories. A chat application (The Freenet Project, 2026b) and an anonymous identity scheme (The Freenet Project, 2026a) are deployed on the live network. Three classes of application fit less well.

Strictly linearizable transactions. A defense against double-spending fungible tokens requires that two attempts to spend the same balance cannot both succeed. The merge model cannot express "one of these updates is rejected unconditionally"; merge is total. Applications that need a strong-consistency primitive must either contain the inconsistency inside a contract whose validity predicate detects and rejects the second update (which forces the conflict into application code, with the loser's state restored on detection) or layer an auxiliary consensus mechanism on top of the platform.

The limitation extends beyond currency. A reputation system that only *accumulates* endorsements is monotone and fits the merge model. Revoking a credential, slashing a stake, or reducing a balance introduces the same requirement to reject a conflicting second update. Applications that otherwise fit the model may therefore require an additional consistency mechanism for non-monotone operations.

Abuse-resistant open posting. A purely open contract is simple to write as a set under union and equally simple to flood. Practical contracts in this class restrict writes to signed records by approved keys, charge a proof-of-work fee per update, or require credentials issued by a reputation contract. The per-neighbor limits of Section 6 cap what one attacker can push through one peer, but they do not stop a distributed flood from many cheap identities or low-rate abuse that stays within per-peer limits, and the platform has no other spam defense.

Durable storage of cold state. Contract state persists only while some peer retains it, and eviction (Section 4) removes the least-demanded contracts first: those with no readers or subscribers. The platform does not recruit a replacement when a host evicts a contract, so a contract nobody reads or subscribes to can disappear from every host, recoverable only by a new PUT from a party that still holds its state. The platform therefore provides no durability guarantee for cold state, and dormant collaborative documents, unpolled inboxes, and identity records of inactive users may stay cold for long periods.

7.4 Open Problems

1. **Scaling characterization.** The measurement in Section 5.6 is consistent with logarithmic-diameter routing at one network size. Characterizing scaling requires measurements across orders of magnitude in N, which the current deployment cannot provide and the simulation harness does not yet produce. The routing estimator's structure was likewise selected in simulation, and the contribution of its per-contract failure term has not yet been established on production traffic.
2. **Enforcing the merge laws.** Containment and conformance testing (Section 4.6) detect some violations, but neither removes a faulty contract from the network. Running conformance testing across the network, propagating its evidence so that other peers can re-verify it, and ceasing to host contracts with confirmed violations remain open. Coverage is also incomplete. A contract that rejects the merge of two individually valid states is reported as inconclusive, although the rejection means the replicas cannot converge, and very large states are not tested. Testing cannot constrain a malicious author, but it gives honest authors a repeatable check.
3. **Congestion control** suited to heterogeneous peer-to-peer links.
4. **Sybil-resistant identity.** Identity and reputation contracts are routed through the location layer they are meant to secure (Section 6). Breaking this circularity requires ring separation among replicas of safety-critical contracts, demonstrable diversity near their locations, or

out-of-band trust anchors. Spreading peers that share an address prefix without restoring the freedom to choose locations is also open.

5. **Incentive and abuse layer.** The network needs a way to resist coordinated denial of service without centralized admission control or excessive barriers to joining.
6. **Durability of cold state.** Maintaining a minimum replica count independently of demand, through active repair, incentivized pinning, or archival contracts, without creating an unbounded free-storage abuse surface.

8 Related Work

Distributed hash tables. Chord (Stoica et al., 2001), Pastry (Rowstron and Druschel, 2001), and Kademlia (Maymounkov and Mazières, 2002) established the structured-overlay model inherited by Freenet's ring. They organize peers by hashed identifiers in a ring or XOR-metric space and provide $O(\log n)$ lookup through maintained routing tables. Freenet stores mutable, contract-defined state at each key and constructs its topology through the mechanism in Section 5, without explicit finger tables.

Content-addressed storage. The original Freenet (Clarke et al., 2001) introduced content-hash keys (CHKs) for immutable data around 1999–2000. IPFS (Benet, 2014) later applied content addressing to web deployment over its Bitswap block exchange. In both systems, external naming or application layers provide mutability. Freenet contracts instead bind mutable state and its consistency rules to content-addressed code.

CRDTs. Conflict-free Replicated Data Types (Shapiro et al., 2011) provide the algebraic foundation for the contract layer. The contract interface implements the state-based (CvRDT) model while allowing applications to supply both the lattice and a summary/delta synchronization protocol. Antidote and AntidoteDB (Akkoorath et al., 2016) similarly provide CRDTs for geo-distributed databases, but expose a predefined CRDT vocabulary.

Bayou. Bayou (Terry et al., 1995) is the closest historical precedent for contracts. In 1995 it used application-supplied merge procedures on trusted replica servers to reconcile concurrent writes; CouchDB, Riak, and Git's merge drivers later also exposed conflict resolution to application code. Bayou procedures had database access and were configured per database. Freenet executes content-addressed merge code as sandboxed WebAssembly on untrusted hosts and restricts it to an idempotent commutative monoid. These restrictions allow replicas to converge without the platform inspecting the merge implementation or trusting its host. Freenet therefore permits more flexibility than fixed-lattice frameworks and less than Bayou.

Anti-entropy and delta sync. Dynamo (DeCandia et al., 2007) and its descendants use Merkle-tree anti-entropy to discover differences between replicas. That is a special case of summary/delta synchronization in which the summary is a hash tree and the delta is the missing subtree; a contract with a more compact summary, such as a list of identifiers or a counter value, needs one round trip rather than $O(\log n)$. `rsync` (Tridgell, 1999) has the same protocol shape at the level of file blocks, without knowledge of application structure. Delta-state CRDTs (Almeida et al., 2018) are the closest formal analogue: they ship only changes between replicas of a fixed-lattice CRDT. In Freenet both the lattice and the delta encoding are supplied by the contract, and synchronization runs over a small-world routing layer instead of a fully connected replica set. Operation-based CRDTs instead propagate individual operations and depend on a

causal-broadcast layer for exactly-once, causally ordered delivery; Freenet propagates state, so it tolerates loss, reordering, and duplication without such a layer.

Local-first software and CRDT libraries. Local-first software (Kleppmann et al., 2019) favors convergence on user devices over server-mediated consistency. Automerge (Automerge contributors, 2026) and Y.js (Y.js contributors, 2026) provide composable CRDT structures such as JSON-like trees, sequences, and counters while leaving transport to the application. Those libraries guarantee convergence when applications remain within their primitives. Freenet lets each contract define its algebra and synchronization protocol, placing the corresponding correctness obligation on the contract author.

Secure Scuttlebutt. SSB (The SSB community, 2026) centers on per-user append-only signed logs replicated by gossip on demand. Such a feed can be represented as a Freenet contract whose merge is set union over signed records. Freenet generalizes the state structure and replication protocol beyond logs while targeting many of the same applications, including private messaging, blogging, and social feeds.

Hypercore. The Hypercore protocol family (Holepunch, 2026) provides content-addressed, append-only signed logs replicated peer-to-peer, with Hyperbee and Hyperdrive adding higher-level data structures. A Hyperbee resembles a contract with library-defined merge semantics, while a local key-management agent over Hypercore serves a role similar to a delegate.

Small-world routing. Kleinberg's result (Kleinberg, 2000) provides the theoretical basis for the routing layer; the original Freenet (Clarke et al., 2001) predated and informally anticipated parts of it. Symphony (Manku et al., 2003) combines short-range ring links with $1/d$ long-range links sampled from a harmonic distribution. Freenet likewise protects its closest successor and predecessor, but continuously adjusts the long-range distribution: an established peer targets an under-represented region of its connections, and the receiver favors candidates that close large gaps. To our knowledge, this combination has not been analyzed in the structured-overlay literature.

Performance-aware routing. Adaptive routing in peer-to-peer overlays has a long history. Freenet uses isotonic regression, a standard calibration method (de Leeuw et al., 2009), as a monotone distance-based prior, and models each neighbor's departure from it, and that neighbor's departure within each region of the key space, as random effects whose weights are estimated from the data (Morris, 1983).

Blockchain platforms. Ethereum (Wood, 2014) and successor smart-contract platforms execute a globally linearized replicated state machine. Freenet instead provides per-contract eventual consistency with application-defined merge. It is intended for interactive applications with mergeable updates and does not support applications that require a single global transaction order.

Federated systems. Matrix (The Matrix.org Foundation, 2026) is a server-based federated system with a similar application surface. Its state-resolution algorithm reconciles divergent room states by a deterministic application-defined rule. Freenet applies this approach in a peer-to-peer setting and permits arbitrary idempotent commutative monoids.

The original Freenet. The original Freenet (Clarke et al., 2001) provided anonymous, content-addressed storage for immutable files. Its heuristic forwarded requests to peers whose previous successful routes most resembled the requested key, anticipating aspects of small-world greedy routing without assigning peers explicit locations. The present system retains content-addressed naming and the small-world routing intuition. It adds explicit locations, contract-defined mutable state, delegates, and user interfaces, while moving anonymity to application-layer protocols.

9 Conclusion

Applications on Freenet define their state algebra in sandboxed WebAssembly. A contract-supplied summary/delta protocol synchronizes that state across a mesh of neighboring co-hosts, while leased subscriptions and demand-ordered eviction keep hosting aligned with active use. The underlying small-world ring protects a nearest successor and predecessor while shaping its remaining links through gap-based CONNECT targeting and Kleinberg-style acceptance scoring. Forwarding ranks neighbors by expected cost, predicted from a distance prior with per-neighbor corrections weighted by the evidence behind them. Because each contract supplies its own merge, the runtime cannot verify the algebraic laws statically; it tests them against observed states, which has already found violations in deployed contracts. Delegates isolate private state and sensitive operations behind a platform-enforced trust boundary.

This architecture targets decentralized group chat, collaborative documents, social feeds, reputation systems, and other applications with mergeable state. Several mechanisms remain unexercised at scale, experimental, or unimplemented. In particular, evaluating asymptotic scaling will require measurements across network sizes rather than further design analysis alone.

Acknowledgements

Special thanks to Ignacio Duart Gomez and Hector Alberto Santos Rodriguez for their substantial engineering contributions to Freenet, and to Steven Starr for his invaluable support with outreach, fundraising, and strategic guidance.

References

Deepthi Devaki Akkoorath, Alejandro Z. Tomsic, Manuel Bravo, Zhongmiao Li, Tyler Crain, Annette Bieniusa, Nuno Preguiça, and Marc Shapiro. Cure: Strong semantics meets high availability and low latency. In *Proceedings of the 36th IEEE International Conference on Distributed Computing Systems (ICDCS)*, pages 405–414, 2016.

Paulo Sérgio Almeida, Ali Shoker, and Carlos Baquero. Delta state replicated data types. *Journal of Parallel and Distributed Computing*, 111:162–173, 2018.

Automerge contributors. Automerge: A CRDT library for building local-first applications, 2026. `https://automerge.org/`, accessed 2026-05-24.

Richard E. Barlow, D. J. Bartholomew, J. M. Bremner, and H. D. Brunk. *Statistical Inference Under Order Restrictions: The Theory and Application of Isotonic Regression.* Wiley, 1972.

Juan Benet. IPFS — content addressed, versioned, P2P file system, 2014. arXiv:1407.3561.

Bytecode Alliance. Wasmtime: A standalone runtime for WebAssembly, 2026. `https://wasmtime.dev/`, accessed 2026-05-24.

Koen Claessen and John Hughes. QuickCheck: A lightweight tool for random testing of Haskell programs. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP)*, pages 268–279, 2000.

Ian Clarke. Renegade: Zero-config nonparametric supervised learning for Rust, 2026. `https://github.com/sanity/renegade`, accessed 2026-08-09.

Ian Clarke, Oskar Sandberg, Brandon Wiley, and Theodore W. Hong. Freenet: A distributed anonymous information storage and retrieval system. In *Designing Privacy Enhancing Technologies: International Workshop on Design Issues in Anonymity and Unobservability*, pages 46–66. Springer, 2001.

Jan de Leeuw, Kurt Hornik, and Patrick Mair. Isotone optimization in R: Pool-adjacent-violators algorithm (PAVA) and active set methods. *Journal of Statistical Software*, 32(5):1–24, 2009.

Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, Gunavardhan Kakulapati, Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Vosshall, and Werner Vogels. Dynamo: Amazon's highly available key-value store. In *Proceedings of the 21st ACM SIGOPS Symposium on Operating Systems Principles (SOSP)*, pages 205–220, 2007.

Bryan Ford, Pyda Srisuresh, and Dan Kegel. Peer-to-peer communication across network address translators. In *USENIX Annual Technical Conference*, pages 179–192, 2005.

Holepunch. Hypercore Protocol: A streaming, append-only, signed log for peer-to-peer applications, 2026. `https://holepunch.to/`, accessed 2026-05-24.

Jon Kleinberg. The small-world phenomenon: An algorithmic perspective. In *Proceedings of the Thirty-Second Annual ACM Symposium on Theory of Computing (STOC)*, pages 163–170, 2000.

Martin Kleppmann, Adam Wiggins, Peter van Hardenberg, and Mark McGranaghan. Local-first software: You own your data, in spite of the cloud. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!)*, pages 154–178. ACM, 2019.

Gurmeet Singh Manku, Mayank Bawa, and Prabhakar Raghavan. Symphony: Distributed hashing in a small world. In *Proceedings of the 4th USENIX Symposium on Internet Technologies and Systems (USITS)*, pages 127–140, 2003.

Petar Maymounkov and David Mazières. Kademlia: A peer-to-peer information system based on the XOR metric. In *International Workshop on Peer-to-Peer Systems (IPTPS)*, pages 53–65. Springer, 2002.

Carl N. Morris. Parametric empirical Bayes inference: Theory and applications. *Journal of the American Statistical Association*, 78(381):47–55, 1983.

Antony Rowstron and Peter Druschel. Pastry: Scalable, decentralized object location, and routing for large-scale peer-to-peer systems. In *Middleware 2001*, pages 329–350. Springer, 2001.

Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. In *Proceedings of the 13th International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS)*, pages 386–400. Springer, 2011.

Ion Stoica, Robert Morris, David Karger, M. Frans Kaashoek, and Hari Balakrishnan. Chord: A scalable peer-to-peer lookup service for internet applications. In *Proceedings of ACM SIGCOMM*, pages 149–160, 2001.

Douglas B. Terry, Marvin M. Theimer, Karin Petersen, Alan J. Demers, Mike J. Spreitzer, and Carl H. Hauser. Managing update conflicts in Bayou, a weakly connected replicated storage system. In *Proceedings of the Fifteenth ACM Symposium on Operating Systems Principles (SOSP)*, pages 172–182, 1995.

The Freenet Project. Ghost Keys: Anonymous, verifiable identities on Freenet, 2026a. `https://freenet.org/ghostkey/`, accessed 2026-05-24.

The Freenet Project. River: A decentralized group chat on Freenet, 2026b. `https://github.com/freenet/river`, accessed 2026-05-24.

The Matrix.org Foundation. The Matrix specification, 2026. `https://spec.matrix.org/`, accessed 2026-05-24.

The SSB community. Secure Scuttlebutt: A protocol for decentralized social applications, 2026. `https://scuttlebutt.nz/`, accessed 2026-05-24.

Tokio Project. Turmoil: A framework for testing distributed systems, 2026. `https://docs.rs/turmoil`, accessed 2026-05-24.

Andrew Tridgell. *Efficient Algorithms for Sorting and Synchronization*. PhD thesis, The Australian National University, 1999.

Gavin Wood. Ethereum: A secure decentralised generalised transaction ledger, 2014. Ethereum Project Yellow Paper.

Y.js contributors. Y.js: Shared data types for building collaborative software, 2026. `https://yjs.dev/`, accessed 2026-05-24.